Context-Aware AI Scaffolding & Agent Architecture

Handbook and starter kit for Claude Code & Google Antigravity / Gemini: token economics, readiness gates, work-order routing, and copy-paste templates to bootstrap any new project. Reference implementation: github.com/f-accomando/bm (agents prefixed *bm-*).

PART A — HOW IT WORKS

1. The Core Principle: Separate Decision-Making from Execution

Frontier models (Claude Opus, Gemini Pro) have a cost bottleneck: **output token asymmetry**. High-tier models cost roughly 20x to 40x more per output token than entry-tier models (Haiku, Flash-Lite), so generating raw implementation code with a top model burns session quotas fast. The fix is structural:

- **High-tier models (Opus / Gemini Pro)** produce *few output tokens*: they plan, design, settle trade-offs and emit structured Work Orders.
- **Low-tier models (Haiku / Gemini Flash)** produce *many output tokens*: they write and refactor code strictly driven by deterministic Work Orders.
- **Readiness routing:** a task is never dispatched by its perceived size or complexity, only by **how well specified** it already is.

2. Session Workflow: Zero Token Waste

Loading whole documents destroys the context budget and breeds hallucinations. The architecture uses hierarchical, on-demand, surgical retrieval:

- **Root guardian** (`CLAUDE.md` / `GEMINI.md`): loaded at start, under 60 lines. Global invariants, branching and license rules, the lookup table, agent routing.
- **Index map** (`docs/progress.md`): the single index. Never read whole; it only routes to the right area sheet.
- **Targeted extraction** with grep and offset/limit reads: about 100 tokens instead of 10,000.

Phase	What the AI does	Files involved / output
1. Boot	Loads base rules and quick commands.	`CLAUDE.md` / `GEMINI.md` (about 50 lines)
2. Routing	Identifies the target area if not obvious.	`docs/progress.md` (lookup only)
3. Lookup	Extracts open tasks and constraints surgically.	`grep -n -A 15 '^## Open work' docs/progress/<area>.md`
4. Execution	Targeted inspection (symbol index) and selective tests.	Specific sources, `make test-<area>`
5. Closing	Leave-no-trace: update state, run the audit.	`docs/progress/<area>.md`, `docs_audit.py`

3. The Agent Architecture: Six Roles

Agent	Claude tier	Antigravity tier	Toolset	Mandate and boundaries
planner	Opus medium	Gemini Pro	Read-only (+ read-only shell)	Turns vague or large tasks into verified Work Orders. **Never writes code.** Always runs before deep.
coder	Haiku high	Gemini Flash	Full (edit, write, exec)	Writes code from a locked Work Order. **No design choices, no ambiguity resolution.**
mechanic	Haiku low	Gemini Flash-Lite	Full (atomic edits)	Zero-risk tasks: symbol renames, formatting, ticking progress/roadmap items.
light	Sonnet medium	Gemini Flash	Full (read, write, exec)	Focused fixes (1-2 files) needing judgment; documentation verified against the real code.
deep	Opus xhigh	Gemini Pro	Full (isolated repro)	Strict gate: one isolated bug with a repro, or one hard architectural decision. Rejects generic tasks.
research	Haiku low	Native research subagent	Read-only (+ read-only shell)	Wide exploration across many files; returns a 5-10 line summary. Never edits.

4. Readiness Gate, Status Protocol and Escalation

Before a task goes to an execution agent (mechanic, coder, light), the router answers five binary questions:

1. **Target files and symbols:** are exact paths and function signatures named?
2. **Diff determinism:** is the change specified so strictly that two developers would write the same diff?
3. **Verifiable command:** is there an automated check (unit test, build, lint, audit script)?
4. **Constrained scope:** does it touch 3 files or fewer?
5. **Zero pending decisions:** is it free of unresolved design choices or dependencies?

Rule: 5 of 5 YES means the task is ready; any NO blocks execution and the request goes to **planner**, which makes it ready or splits it into atomic Work Orders. Never send an unprepared task straight to **deep**: it only accepts a symptom, an exact reproduction, a suspect area, and the attempts already made, and otherwise answers `REJECTED` and bounces back to planner. Every agent ends with exactly one status:

Status	Meaning	Next step
`DONE`	Verification command passed; short diff summary.	Close the task (section 19, rule R9).
`BLOCKED: <reason>`	Ambiguous spec, mismatched signature or missing decision.	Back to planner.
`FAILED x2`	Verification still fails after two fix attempts.	Escalate one tier (coder to light, light to deep).

5. Standard Work Order Template

```
### WORK ORDER: [WO-ID]
- Objective: [one concise sentence defining the outcome]
- Target Files & Symbols: [explicit paths and functions, e.g. src/module/file.c:init_thing()]
- Required Mutation: [unambiguous diff specification leaving zero design latitude]
- Verification Command: [deterministic command, e.g. make test-<area> or pytest -k <name>]
- Expected Result: [exit code 0 and the exact expected assertion/output]
- Designated Agent: [mechanic | coder | light]
```

6. Three-Tier Repository Structure

To prevent context rot, documentation is split across three lifecycles:

- **Root guardians** (`CLAUDE.md`, `GEMINI.md`): under 60 lines. Only global invariants, build shortcuts, branching rules, agent routing. No release history.
- **Living progress** (`docs/progress/<area>.md`): active sheets of 80-120 lines: current state, open work, binding local rules.
- **Deep RFCs** (`docs/design/<topic>.md`): speculative designs and long-term proposals, kept out of the live index so they never inflate token cost.

PART B — STARTER KIT FOR A NEW PROJECT (COPY, FILL THE <PLACEHOLDERS>, COMMIT)

7. Bootstrap Checklist and File Tree

1. Create the tree below and copy the templates of sections 8-14 into it.
2. Fill `CLAUDE.md`: stack, test commands, global invariants. Keep it under 60 lines.
3. Copy the six agents (section 10); adjust the `tools` list and the verification commands to your stack.
4. Copy the three skills (section 11) and `.claude/settings.json` (section 12).
5. Create `docs/progress.md` and one first area sheet (section 13); add `scripts/docs_audit.py` (section 14).
6. Wire the audit into the pre-commit hook, the Makefile and CI (section 15). Run it once and save the baseline score.
7. Create `GEMINI.md` if you also use Antigravity (section 9). Commit, then hand the first real task to planner to test the gate.

```
<project>/
|-- CLAUDE.md                   root guardian, under 60 lines (loaded every session)
|-- GEMINI.md                   same sections 1,2,4,5 + Antigravity routing table
|-- AGENTS.md                   optional: roles, gate, Work Order format (shared governance)
|-- .claude/
|   |-- settings.json           skill overrides, permissions allow/deny
|   |-- agents/                 planner, coder, mechanic, light, deep, research (.md)
|   `-- skills/
|       |-- work-order/SKILL.md
```

```
|       |-- docs-map/SKILL.md
|       `-- docs-loop/SKILL.md
|-- .agents/skills/              mirror of .claude/skills for Antigravity
|-- .githooks/pre-commit         versioned hook (git config core.hooksPath .githooks)
|-- .github/workflows/ci.yml     includes the docs-audit step
|-- docs/
|   |-- progress.md              the single index
|   |-- progress/<area>.md       living sheets, 80-120 lines each
|   |-- design/<topic>.md        RFCs
|   `-- ROADMAP.md               optional milestones
`-- scripts/
    |-- docs_audit.py            score: tokens + penalties (section 14)
    `-- cindex.py                optional symbol/banner index for big source files
```

8. Template: CLAUDE.md (root guardian)

Claude Code concatenates every `CLAUDE.md` it finds from the working directory upward and loads it in full each session. The official guidance is to stay under 200 lines; this kit is deliberately stricter (60) because the file is paid for on every turn. Treat it as context, not enforcement: anything that must *never* happen belongs in a permission rule or a hook (section 12).

CLAUDE.md

```
# <Project> - Session Notes (Claude & Gemini)
Repo <org>/<repo>, main branch `main`. Stack: <languages, framework, OS/targets>.
Single state index: docs/progress.md. Never read a file whole: grep first, then Read with offset/limit.

## 1. Targeted lookup (zero token waste)
| Need                           | Command                                                          |
|--------------------------------|------------------------------------------------------------------|
| What is left in an area        | grep -n -A 15 '^## Open work' docs/progress/<area>.md            |
| Binding rules of an area       | grep -n -A 20 '^## Rules' docs/progress/<area>.md                |
| Find a symbol in a big file    | python scripts/cindex.py <file> <symbol>  (then Read offset)     |
| Design background              | docs/design/<topic>.md (only when asked)                         |

## 2. Rules of engagement
- Branch: propose a short name (feat/<name>) and ask before editing; start from up-to-date main.
- Tests: run only the tests of the touched area (make test-<area>); never the whole suite.
- No secrets (keys, tokens, .env) in the repo. Commits only when the user asks.
- <License / dependency constraints, e.g. no GPL code copied into the sources>.

## 3. Subagent routing (definitions in .claude/agents/)
Route by how well a task is specified, not by its size.
| Agent     | Model / effort        | Does                                          | Never                  |
|-----------|-----------------------|-----------------------------------------------|------------------------|
| planner   | opus medium, read-only| vague/large task -> Work Orders               | writes code            |
| coder     | haiku high            | code from a locked Work Order                 | design decisions       |
| mechanic  | haiku low             | renames, format, progress ticks               | any decision           |
| light     | sonnet medium         | 1-2 file fixes, docs checked against code     | long or vague work     |
| deep      | opus xhigh            | one isolated bug with repro / one decision    | whole features         |
| research  | haiku low, read-only  | wide search, 5-10 line summary                | edit files             |
Readiness gate (5/5 YES for coder, mechanic, light): files+symbols named; diff deterministic; verify
command exists; <= 3 files; no pending decisions. Any NO -> planner. Never straight to deep.
Every agent ends with: DONE | BLOCKED: <reason> | FAILED x2 (escalate one tier).

## 4. Global invariants
- <Invariant 1, e.g. the public API of module X is frozen>
- Area-specific rules live in docs/progress/<area>.md under "## Rules (do not break)".

## 5. Closing a task (leave no trace)
1. Update "## Open work" in the area sheet (remove or tick finished items).
2. New constraint or pattern? Add it under "## Rules (do not break)".
3. Run python scripts/docs_audit.py score: no broken links, no orphans, score not higher than baseline.
```

9. Templates: GEMINI.md and AGENTS.md (cross-agent parity)

Claude Code reads `AGENTS.md` on its own **only when no CLAUDE.md exists** in the working directory or above; with both present it loads CLAUDE.md alone, unless CLAUDE.md imports it with `@AGENTS.md` (imports are expanded at launch, so they do not save tokens). Do not assume AGENTS.md is auto-loaded next to a CLAUDE.md. Parity rule: `GEMINI.md` repeats sections 1, 2, 4 and 5 of CLAUDE.md verbatim and swaps only the routing table for the Antigravity mapping below.

GEMINI.md (section 3 replaced; all other sections identical to CLAUDE.md)

```
# <Project> - Session Notes (Google Antigravity / Gemini)
...sections 1 and 2 copied verbatim from CLAUDE.md...

## 3. Subagent routing (Antigravity invoke_subagent)
| Role      | Model          | Toolset                 | Does / never                              |
|-----------|----------------|-------------------------|-------------------------------------------|
| planner   | "pro"          | read-only               | Work Orders / never writes code           |
| coder     | "flash"        | full                    | code from Work Order / never designs      |
| mechanic  | "flash_lite"   | full, atomic edits      | zero-risk edits / never decides           |
| light     | "flash"        | full                    | 1-2 file fixes, verified docs             |
| deep      | "pro"          | full                    | one isolated bug with repro               |
| research  | native         | read-only               | wide search, short summary                |
Readiness gate, status line and escalation: identical to CLAUDE.md section 3.

...sections 4 and 5 copied verbatim from CLAUDE.md...
```

AGENTS.md (optional shared governance; link it, do not duplicate it elsewhere)

```
# <Project> - Agent Governance
1. Principle: top models decide (few tokens), cheap models execute (many tokens). Route by readiness.
2. Roles table: planner, coder, mechanic, light, deep, research (model, toolset, mandate, limits).
3. Readiness gate (5 questions) and the rule: any NO -> planner.
4. deep entry gate: symptom + exact repro + suspect area + attempts already made, else REJECTED.
5. Closing statuses: DONE | BLOCKED: <reason> | FAILED x2.
6. Work Order format (section 5 of this guide).
```

10. Templates: the six agents (.claude/agents/<name>.md)

Project agents live in `.claude/agents/` and override user-level agents of the same name. Required frontmatter: `name`, `description` (the description is what the router reads, so state *when* to use the agent). Useful optional fields: `tools` (allow-list; omit to inherit all), `disallowedTools`, `model` (`haiku`, `sonnet`, `opus`, a full model ID, or `inherit`), `effort` (`low` to `max`), `maxTurns`, `skills`, `memory`. Unknown fields are silently ignored. **Important:** `tools` is an allow-list, so an agent told to run a verification command must list `Bash`.

.claude/agents/planner.md

```
---
name: planner
description: Use for vague, large or under-specified tasks: produces Work Orders, never code. Call before deep.
model: opus
effort: medium
tools: Read, Grep, Glob, Bash
---
You are the planning agent. Your output is short and decisive: plans and decisions, never code.

## Mandate
1. Turn vague or large requests into atomic, self-contained Work Orders.
2. Name the exact files and symbols first (grep, symbol index, Read with offset/limit).
3. A requirement is missing? Do not guess: list it as an explicit question for the user.

## Hard limits
- No implementation code and no long diffs. Bash only for read-only commands (grep, git log/diff, cindex).
- Compact output: one Work Order block per atomic step, in the Work Order format of AGENTS.md.
```

.claude/agents/coder.md

```
---
name: coder
description: Writes code only from a precise, complete Work Order. No design decisions, no guessing.
model: haiku
effort: high
tools: Read, Edit, Write, Grep, Glob, Bash
---
You write code from locked Work Orders and change nothing else.

## Rules
1. No open design: a choice left open or a missing parameter means stop and reply
   `BLOCKED: decision needed - <detail>`. Never invent a solution.
2. Minimal diff: touch only the files and symbols named in the order. No side refactors.
3. Verify: run the order's verification command before finishing.

## Closing status (exactly one)
- DONE: verification passed; 2-3 line diff summary.
- BLOCKED: <reason>: ambiguous spec or missing decision; back to planner.
- FAILED x2: still failing after two fix attempts; report the error, escalate to light or deep.
```

.claude/agents/mechanic.md

```
---
name: mechanic
description: Fully specified zero-risk edits: renames, formatting, templates, progress ticks. No decisions.
model: haiku
effort: low
tools: Read, Edit, Write, Grep, Glob, Bash
---
You execute zero-risk, fully specified, purely mechanical tasks.

## Rules
1. Zero decisions: do exactly what was asked. No business-logic changes, no invented tasks, no reprioritising.
2. Integrity: keep the existing structure of progress docs; after a rename run the build/lint command
   and make sure no syntax error was introduced.
3. Close with DONE (1-2 line summary) or BLOCKED (instruction ambiguous or exact coordinates missing).
```

.claude/agents/light.md

```
---
name: light
description: Small tasks needing judgment: code questions, 1-2 file fixes, docs verified against real code.
model: sonnet
effort: medium
tools: Read, Edit, Write, Grep, Glob, Bash
---
You handle focused tasks and verified documentation.

## Code
- Smallest change that fixes the problem; no wide refactors.
- Always run the specific test or verification command for the change.
- Fix still failing after two attempts: return `FAILED x2` and escalate to deep.

## Documentation
- Zero hallucination: read the real code before documenting an API or a behaviour.
- Match the style and format of the existing docs; do not touch sources except requested comments.

## Closing: DONE | BLOCKED (needs an architectural decision) | FAILED x2
```

.claude/agents/deep.md

```
---
name: deep
description: ONE isolated hard problem with a sure repro (or one critical design decision). Never whole features.
model: opus
effort: xhigh
tools: Read, Edit, Write, Grep, Glob, Bash
---
You are the highest-depth agent: costly, so you act only on isolated problems with objective evidence.

## Entry gate - accept ONLY if the task contains all four
1. Clear symptom.  2. Exact deterministic reproduction (command, input or failing test).
3. Suspect area (file, module, subsystem).  4. Attempts already made and ruled out.
Otherwise reply: `REJECTED: back to planner - missing entry requirements (repro / isolated problem)`.

## Mandate
- Root-cause analysis BEFORE touching any line; then a minimal, targeted fix.
- Large implementation or rewrites go back to coder or light through a Work Order.
- Explain the factual evidence for the cause before applying the fix; never repeat failed attempts.
- Run the reproduction to certify the fix; close with DONE and a report (symptom, cause, minimal fix).
```

.claude/agents/research.md

```
---
name: research
description: Read-only wide exploration of many files; answers with compact summaries. Never edits.
model: haiku
effort: low
tools: Read, Grep, Glob, Bash
---
You are the read-only research subagent.

## Rules (zero token waste)
1. Read-only: no edits, no new files. Bash only for read-only commands (grep, git log, cindex).
2. Navigate efficiently: grep -n and the symbol index for exact positions; never read huge files whole,
   use Read with small offset and limit.
3. Ultra-compact answer: no pasted code blocks; at most 5-10 bullets giving file:line paths,
   function signatures or data layouts, and a one-line conclusion for the calling agent.
```

11. Templates: three skills (.claude/skills/<name>/SKILL.md)

A skill is a folder with a `SKILL.md`. Only its **description** sits in context permanently; the body loads when the skill is invoked (progressive disclosure), so keep descriptions to one or two sentences with the trigger first, and bodies short (the official ceiling is 500 lines; this kit uses under 60). Claude Code reads `.claude/skills/`; Antigravity reads `.agents/skills/`, so mirror the folder if you use both.

.claude/skills/work-order/SKILL.md

```
---
name: work-order
description: Builds Work Orders and applies the readiness gate before delegating to coder, mechanic or light.
---
# Work Order and readiness gate
Answer the 5 questions: (1) files and symbols named? (2) diff deterministic? (3) verify command exists?
(4) 3 files or fewer? (5) no pending decisions?
- 5/5 YES: task READY -> choose mechanic, coder or light.
- any NO: NOT READY -> planner investigates and emits one or more Work Orders.

## Template
### WORK ORDER: [ID]
- Objective / Target Files & Symbols / Required Mutation / Verification Command / Expected Result / Designated Agent

## Closing protocol
DONE (verification passed) | BLOCKED: <reason> (back to planner) | FAILED x2 (escalate one tier)
```

.claude/skills/docs-map/SKILL.md

```
---
name: docs-map
description: Which file to open per area and how to read big files cheaply. Use before reading docs or large files.
---
# Where to read and how to update (little, in sections)
One index: docs/progress.md. Start there. Never read a big file whole: grep -n the symbol or '^##',
then Read with offset/limit. A file already read in this session is not read again.

## Area map
| Area   | Open                       |
|--------|----------------------------|
| <area> | docs/progress/<area>.md    |

## Updating progress after a change
1. grep -n '^##' the area sheet; read only the subsection of the module you touched.
2. Fix or delete lines that became false. Record state (files, symbols), never dates or "we did X".
3. Sheet over ~120 lines: split by topic (<area>-<topic>.md), link it from the parent and the index.
4. Delegate to mechanic when the lines are already decided.

## Wide searches
"Where is X / who uses Y" across many files -> research agent (returns the conclusion, not the files).
```

.claude/skills/docs-loop/SKILL.md

```
---
name: docs-loop
description: Tidies CLAUDE.md and docs/progress* in measure-keep-discard rounds. Use after doc edits or to cut tokens.
---
# docs-loop
Measure = python scripts/docs_audit.py score (lower is better).

## Loop (max 8 rounds, one hypothesis per round), pick in this order
1. An audit penalty: orphan -> link it; broken -> fix; over 120 lines -> split; duplicate -> keep one copy.
2. An area-specific rule sitting in CLAUDE.md -> move it to the area sheet "Rules", leave one line.
3. The most re-read file -> split it or add a search command for it to the docs-map skill.
4. An index row that does not help find a file -> remove it.
Each round: back up touched files; edit; check information was MOVED, not lost; run score.
Lower and nothing lost -> keep and log it; otherwise restore the backup and log a discard.
Stop at 8 rounds, after 2 discards in a row, or when out of hypotheses.

## Repeated workflows become skills
A prompt or command sequence seen in 3 or more sessions, covered by no skill, becomes a skill
(under 60 lines, trigger in the description).

## Close
Report score before -> after, rounds kept/discarded, skills created. No commit unless asked.
```

12. Template: .claude/settings.json

Two jobs: keep the always-loaded context small, and turn the rules that must never be broken into **enforcement** rather than advice. Permission rules use the form `Tool(pattern)`; for logic a pattern cannot express, use a `PreToolUse` hook. Project settings go in `.claude/settings.json` (shared); personal overrides in `.claude/settings.local.json` (git-ignored). The `skillOverrides` key comes from the reference project and is not in the public settings reference I checked: verify it on your Claude Code version.

.claude/settings.json

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Bash(make test-*)",
      "Bash(python3 scripts/docs_audit.py *)",
      "Bash(python3 scripts/cindex.py *)"
    ],
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)"
    ]
  },
  "skillOverrides": {
    "anthropic-skills:docx": "off",
    "anthropic-skills:pptx": "off",
    "anthropic-skills:xlsx": "off"
  }
}
```

13. Templates: progress index and area sheet

docs/progress.md (the only index; relative links, so the audit can follow them)

```
# <Project> - Progress Index
State, not history. Open ONE area sheet, never all of them.

## Which section to open
| Area   | Sheet                           | Open it when                        |
|--------|---------------------------------|-------------------------------------|
| <area> | [core](progress/<area>.md)      | <one line: what lives there>        |

## Open milestones
- M1 <name>: <one line> (details: [ROADMAP](ROADMAP.md))

## Reference docs
- [design/<topic>.md](design/<topic>.md): speculative RFCs, not state.
```

docs/progress/<area>.md (80-120 lines; three fixed headings)

```
# <Area> - current state
## How it is today
### <module>
- <what exists now>; symbols to search: `file.ext:symbol`

## Open work
- [ ] <task>. Verify: `<command>`

## Rules (do not break)
- <binding constraint>. Why: <one line>
```

14. Starter: scripts/docs_audit.py

A dependency-free audit (Python 3). The score is the tokens always loaded (`CLAUDE.md` + index) plus the mean area-sheet size plus penalties; it exits non-zero on any penalty or if the score rises above the saved baseline. `score --save` writes the baseline to `.docs-score`.

scripts/docs_audit.py

```
#!/usr/bin/env python3
"""docs_audit.py score [--save]: one number (lower is better). Tokens = bytes / 4."""
import glob, os, re, sys
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
INDEX, SHEET_MAX, ROOT_MAX = "docs/progress.md", 120, 60
BASE = os.path.join(ROOT, ".docs-score")

def rd(p):
    try:
        return open(os.path.join(ROOT, p), encoding="utf8").read()
    except OSError:
        return ""

def tok(p):
    return len(rd(p).encode()) / 4

def links(p):
    for m in re.finditer(r"\]\(([^)#\s]+\.md)", rd(p)):
        yield os.path.normpath(os.path.join(os.path.dirname(p), m.group(1))).replace(os.sep, "/")

def score(save):
    sheets = sorted(os.path.relpath(f, ROOT).replace(os.sep, "/") for f in glob.glob(ROOT + "/docs/progress/*.md"))
    linked, pen = set(links(INDEX)), []
    if rd("CLAUDE.md").count("\n") > ROOT_MAX:
        pen.append((400, f"CLAUDE.md over {ROOT_MAX} lines: move area rules to their sheet"))
    if INDEX not in rd("CLAUDE.md"):
        pen.append((2000, "CLAUDE.md does not point to the single index"))
    for s in sheets:
        if s not in linked:
            pen.append((500, f"orphan: {s}"))
        if rd(s).count("\n") > SHEET_MAX:
            pen.append((200, f"{s} over {SHEET_MAX} lines: split by topic"))
    for f in ["CLAUDE.md", INDEX] + sheets:
        for l in links(f):
            if not os.path.exists(os.path.join(ROOT, l)):
                pen.append((300, f"broken link in {f}: {l}"))
    mean = sum(map(tok, sheets)) / max(1, len(sheets))
    total = round(tok("CLAUDE.md") + tok(INDEX) + mean + sum(p for p, _ in pen))
    print(f"score {total}")
    for _, m in pen:
        print("  !", m)
    bad = bool(pen)
    if os.path.exists(BASE) and not save:
        base = int(open(BASE).read().strip() or 0)
        if total > base:
            print(f"  ! score rose above baseline {base}")
            bad = True
    if save and not bad:
        open(BASE, "w").write(str(total))
    return 1 if bad else 0

if __name__ == "__main__":
    a = sys.argv[1:]
    sys.exit(score("--save" in a) if a[:1] == ["score"] else __doc__)
```

14b. Starter: scripts/cindex.py (symbol index for big files)

Lets an agent list sections and function/class names with line numbers, then read only the needed range with offset/limit. Supports C/C++, Python, JS/TS, Go, Rust and Lua. Usage: python3 scripts/cindex.py [keyword]. Check: running it on itself lists def main.

scripts/cindex.py (chmod +x scripts/cindex.py)

```
#!/usr/bin/env python3
"""Index of a large source file to allow surgical reading:
  python3 scripts/cindex.py FILE [FILTER_KEYWORD]

Extracts:
- Section banners / header comments (e.g. /* ---- Section ---- */ or # ---- Section ----)
- Function and class definitions (C, C++, Python, TypeScript, JavaScript, Go, Rust, Lua)
- Line numbers for precise range-reading with offset/limit.
"""
import re
import signal
import sys

if hasattr(signal, "SIGPIPE"):
    signal.signal(signal.SIGPIPE, signal.SIG_DFL)

PATTERNS = {
    "c_cpp": re.compile(
        r"^(?:static\s+|inline\s+|virtual\s+)*[A-Za-z_][\w\s\*&:<>,]*?\b([A-Za-z_]\w*)\s*\([^;]*$"),
    "python": re.compile(r"^\s*(def|class)\s+([A-Za-z_]\w*)"),
    "js_ts": re.compile(
        r"^\s*(?:export\s+)?(?:async\s+)?(?:function\s+([A-Za-z_]\w*)"
        r"|(?:class|interface|type)\s+([A-Za-z_]\w*)"
        r"|const\s+([A-Za-z_]\w*)\s*=\s*(?:async\s*)?\()"),
    "go": re.compile(r"^\s*func\s+(?:\([^)]+\)\s+)?([A-Za-z_]\w*)\s*\("),
    "rust": re.compile(r"^\s*(?:pub\s+)?(?:async\s+)?fn\s+([A-Za-z_]\w*)"),
    "lua": re.compile(r"^\s*(?:local\s+)?function\s+([\w.:]+)|^\s*([\w.]+)\s*=\s*function"),
    "banner": re.compile(r"^(?:/\*|#|//)\s*-{3,}\s*(.*?)\s*(?:-{3,}|\*/)?$")
}

def main():
    if len(sys.argv) < 2:
        sys.exit(__doc__)
    path = sys.argv[1]
    keyword = sys.argv[2].lower() if len(sys.argv) > 2 else None

    out = []
    total_lines = 0

    try:
        with open(path, "r", encoding="utf-8", errors="replace") as f:
            for total_lines, line in enumerate(f, 1):
                line_str = line.rstrip("\n")
                # Banner Detection
                m_ban = PATTERNS["banner"].match(line_str)
                if m_ban and m_ban.group(1).strip():
                    out.append((total_lines, f"## {m_ban.group(1).strip()}"))
                    continue

                # Code Signature Detection
                sym = None
                if path.endswith((".c", ".h", ".cpp", ".hpp")):
                    m = PATTERNS["c_cpp"].match(line_str)
                    if (m and not line_str.startswith((" ", "\t", "#", "}", "/"))
                            and m.group(1) not in ("if", "while", "for", "switch", "return")):
                        sym = m.group(1)
                elif path.endswith(".py"):
                    m = PATTERNS["python"].match(line_str)
                    if m:
                        sym = f"{m.group(1)} {m.group(2)}"
                elif path.endswith((".js", ".ts", ".jsx", ".tsx")):
                    m = PATTERNS["js_ts"].match(line_str)
                    if m:
                        sym = m.group(1) or m.group(2) or m.group(3)
                elif path.endswith(".go"):
```

```
                    m = PATTERNS["go"].match(line_str)
                    if m:
                        sym = f"func {m.group(1)}"
                elif path.endswith(".rs"):
                    m = PATTERNS["rust"].match(line_str)
                    if m:
                        sym = f"fn {m.group(1)}"
                elif path.endswith(".lua"):
                    m = PATTERNS["lua"].match(line_str)
                    if m:
                        sym = m.group(1) or m.group(2)
                if sym:
                    out.append((total_lines, f"  {sym}"))
    except FileNotFoundError:
        sys.exit(f"Error: file not found: {path}")

    for ln, txt in out:
        if keyword and keyword not in txt.lower():
            continue
        print(f"{ln:6d}  {txt}")
    print(f"-- {total_lines} lines; Read with offset/limit on the range needed", file=sys.stderr)

if __name__ == "__main__":
    main()
```

14c. Starter: per-area test targets in the Makefile

Agents run only the tests of the touched area (section 15B adds the docs target next to these). Recipe lines MUST start with a TAB, not spaces (otherwise: missing separator). To change areas, edit AREAS and create tests//. Check: make test-core.

Makefile

```
AREAS := core api utils cli

.PHONY: test test-all $(addprefix test-, $(AREAS))

test: test-core            # default target: fast smoke test

test-core:                 # one target per area; the recipe line starts with a TAB
■@echo "==> [TEST] Area: core"
■@pytest tests/core -q --tb=short || npm test -- --grep "core"
test-api:
■@echo "==> [TEST] Area: api"
■@pytest tests/api -q --tb=short || npm test -- --grep "api"

# test-utils and test-cli: same shape, change the area name

test-all: $(addprefix test-, $(AREAS))     # CI or before a tag/merge only
```

PART C — AUTOMATION AND TOOLING

15. The Three Pillars of Continuous Automation

A. Git pre-commit hook. Hooks inside `.git/hooks` are not versioned, so keep the hook in the repo and point git at it once per clone:

.githooks/pre-commit (chmod +x, then: git config core.hooksPath .githooks)

```
#!/bin/sh
python3 scripts/docs_audit.py score || { echo "docs audit failed: fix links/orphans/size"; exit 1; }
```

B. Makefile / build tool. Add a docs target and include it in the standard test target:

```
test-docs:
    python3 scripts/docs_audit.py score    # recipe lines must start with a TAB, not spaces

test: test-docs   # add to the existing prerequisites of your standard test target
```

C. CI/CD (GitHub Actions). A dedicated step inside the build/test workflow that runs on pull requests:

```
      - name: Check documentation audit
        run: python3 scripts/docs_audit.py score
```

16. Focus: Google Antigravity & Gemini

Claude relies on `CLAUDE.md`; Google Antigravity uses a modular, enterprise-style architecture:

- **Primary file `GEMINI.md` (or `AGENTS.md`):** takes absolute precedence. It has a dedicated rules budget (20,000 tokens, separate from general context) with smart truncation and a switch to on-demand pointers when exceeded.
- **Native progressive disclosure:** skills in `.agents/skills/` (or `.claude/skills/`) expose only a short name and description in the prompt; the long body loads only when the model decides to activate the skill.
- **Mobile / cloud relay via GitHub Actions:** with `.github/workflows/antigravity-agent.yml` the agent can be launched from a smartphone on GitHub Actions runners, using no local resources.
- **Parallel asynchronous subagents** with differentiated tools (for example a read-only `research` agent that keeps the main context clean, and independent self-contained runs).

Aspect	Claude Code	Google Antigravity
Root instructions	`CLAUDE.md` (AGENTS.md only if CLAUDE.md is absent or imported)	`GEMINI.md` or `AGENTS.md`
Skills folder	`.claude/skills/<name>/SKILL.md`	`.agents/skills/` (also reads `.claude/skills/`)
Agent definitions	`.claude/agents/*.md` (model alias, effort, tools)	invoke_subagent with model "pro", "flash" or "flash_lite"
Hard enforcement	permissions allow/deny, hooks	per-runtime settings; keep the same rules in GEMINI.md

16b. Starter: .github/workflows/antigravity-agent.yml

Workflow that runs the agent headless or remotely through workflow_dispatch. The three inputs are filled by the remote client (leave them empty when launching by hand).

.github/workflows/antigravity-agent.yml

```
name: Antigravity Agent

on:
  workflow_dispatch:
    inputs:
      pair:
        description: "Pairing identifier for remote agent connection"
        required: false
      token:
        description: "One-time session token"
        required: false
      expires:
        description: "Session expiry timestamp (unix epoch)"
        required: false

permissions:
  contents: write
  pull-requests: write

concurrency:
  group: antigravity-agent
  cancel-in-progress: true

jobs:
  agent:
    runs-on: ubuntu-latest
    timeout-minutes: 60

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Set up runtime environment
        uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Install agent CLI
        run: |
          curl -fsSL --retry 5 --retry-all-errors --retry-delay 2 https://antigravity.google/cli/install.sh | bash
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"

      - name: Configure git author
        run: |
          git config user.name "Agentic Assistant"
          git config user.email "assistant-bot@users.noreply.github.com"

      - name: Restore credentials
        env:
          AUTH_TOKEN: ${{ secrets.AGENT_AUTH_TOKEN }}
        run: |
          if [ -n "$AUTH_TOKEN" ]; then
            umask 077
            mkdir -p "$HOME/.gemini/antigravity-cli"
            printf '%s' "$AUTH_TOKEN" | base64 -d > "$HOME/.gemini/antigravity-cli/antigravity-oauth-token"
          fi

      - name: Run agent session
        env:
          AGENT_RELAY: ${{ secrets.AGENT_RELAY }}
          AGENT_PAIR_ID: ${{ inputs.pair }}
          AGENT_PAIR_TOKEN: ${{ inputs.token }}
          AGENT_PAIR_EXPIRES: ${{ inputs.expires }}
        run: |
          if [ -n "$AGENT_RELAY" ]; then
            echo "Starting agent session with relay..."
            agy agent --relay "$AGENT_RELAY"
          else
            echo "Starting standalone agent task..."
            agy agent
          fi
```

1. The file must be on the default branch for workflow_dispatch to appear under Actions.

2. Repository secrets (Settings > Secrets and variables > Actions): AGENT_AUTH_TOKEN = the agent's OAuth token file encoded with base64 -w0 (optional); AGENT_RELAY = relay URL for the remote session (optional; empty means standalone). Never write these values in the repo.
3. The install URL and the agy command belong to the Antigravity CLI: with another agent, replace the steps Install agent CLI and Run agent session. Drop the write permissions if the agent only reads.

17. Claude Plugins: crab-ledger and better-prompt

Two open-source plugins extend the efficiency and quality of Claude sessions:

- **crab-ledger (side pane with cost and efficiency):** shows where tokens, time and money go and whether you are getting more efficient (an animated pixel crab keeps the score). It reads the session logs Claude Code already writes in `~/.claude/projects` and asks git for the lines added and deleted: nothing leaves your machine. Open it with `/ledger`; needs node and git on the PATH. It writes `~/.claude/crab-ledger-report.json`, which `scripts/docs_audit.py rereads` uses to find files re-read so often that they need splitting.
- **better-prompt (session watcher):** warns when the conversation drifts across topics and shows how much context and 5h/7d allowance you have used. It suggests, as plain text, /clear, /compact or /rewind and never runs them; each prompt of 20+ characters costs one small call to a cheap model (haiku by default). Settings in `/config`.

Install both from a terminal Claude Code session (`/plugin` is not available in the desktop app Code tab, but a mod installed at user scope loads there too). Answer y to add the marketplace, then pick a scope:

```
/plugin install crab-ledger --marketplace f-accomando/crab-ledger
/plugin install better-prompt --marketplace f-accomando/better-prompt

Try from a clone without installing:  claude --plugin-dir /path/to/crab-ledger
```

Both are optional: without them builds, tests and scripts work unchanged. The docs-loop skill reads the crab-ledger report to decide which files to split first.

18. What Claude Can Inherit from Antigravity / Gemini

1. **Separate research subagent for heavy searches:** instead of reading dozens of files in the main session, delegate to a fast read-only agent (Haiku/Sonnet) that returns a 5-line summary.
2. **Progressive disclosure for instructions:** move complex procedures into on-demand skills with YAML frontmatter and keep `CLAUDE.md` to the bare minimum.
3. **Cross-agent deduplication:** keep `CLAUDE.md` and `GEMINI.md` aligned on the same index (`docs/progress.md`) so any team member can use either assistant with the same guarantees.

18b. Push refused (HTTP 403) and opening the pull request

If git push fails with 403 the commits are still in the local branch. From the user's terminal (branch naming follows the agent/ invariant; gh auth login first if needed):

```
git branch -M agent/<feature>
git push -u origin agent/<feature>

gh pr create --base main --head agent/<feature> \
  --title "chore: <title>" --body "<one-line summary>"
```

- The token or GitHub App needs Repository: Contents (Read and Write) and Pull Requests (Read and Write).
- If the push to main is rejected, check the branch protection rules: work on a branch and open a pull request.

PART D — BASELINE RULES TO IMPROVE AND IMPLEMENT THIS GUIDE

19. Baseline Rules and the Improvement Loop

Start every new project with these rules. Each one names how it is enforced, so the scaffolding improves itself instead of decaying.

#	Rule	Enforced by
R1	**One index.** All state is reached from `docs/progress.md`; CLAUDE.md points there and lists no other doc.	Audit penalty (+2000)
R2	**Thin root.** CLAUDE.md under 60 lines; a rule that concerns one area moves to that area's Rules section, leaving one line.	Audit penalty (+400)
R3	**State, not chronicle.** Sheets say how things are now: files and symbols, no dates, no "we did X". Closed work is removed from Open work.	docs-loop review

#	Rule	Enforced by
R4	**Small sheets.** 80-120 lines each; larger ones are split by topic and linked from parent and index.	Audit penalty (+200)
R5	**Surgical reads.** Never read a big file whole; grep then offset/limit; never re-read a file already read in the session; wide searches go to research.	crab-ledger re-read report
R6	**Route by readiness, not size.** Apply the 5-question gate; any NO goes to planner; nothing reaches deep without a repro.	Agent descriptions, work-order skill
R7	**Explicit status.** Every agent ends with DONE, BLOCKED or FAILED x2; at most two attempts per tier, then escalate one tier.	Agent bodies
R8	**Verification is mandatory.** No Work Order without a deterministic command; agents that must run it get Bash in `tools`.	Work Order template, agent frontmatter
R9	**Leave no trace.** Close each task: update Open work, add any new constraint to Rules, run the audit.	CLAUDE.md section 5, hook
R10	**Ratchet.** The audit score never rises above the saved baseline; lower it deliberately and re-save.	docs_audit.py, CI
R11	**Improvement loop.** One hypothesis per round, measure, keep or discard, maximum 8 rounds, stop after 2 consecutive discards.	docs-loop skill
R12	**Repeat means skill.** A workflow seen in 3 or more sessions becomes a skill under 60 lines with the trigger in its description.	docs-loop skill
R13	**Learn from cost.** A bug or surprise that cost more than one session adds one line to the area's Rules, with the reason.	Closing step (R9)
R14	**Tune from evidence.** Monthly, count BLOCKED and FAILED x2 per agent: many BLOCKED means the gate or planner is too weak; many FAILED x2 at one tier means raise the tier or tighten the Work Order template.	Manual review
R15	**Hard rules are enforced, not requested.** Secrets, destructive commands and forbidden paths go in permission deny rules or a PreToolUse hook, not only in CLAUDE.md.	settings.json
R16	**Always-loaded budget.** Turn off built-in skills the project never uses; keep every skill description to 1-2 sentences.	settings.json, review
R17	**Parity.** CLAUDE.md and GEMINI.md share sections 1, 2, 4, 5 verbatim; only the routing table differs.	Pull-request review
R18	**Safe defaults.** Branch before editing, selective tests only, no secrets in the repo, commits only on request.	CLAUDE.md section 2

PART E — WORKED EXAMPLE: THEORETICAL ESTIMATE VS LIVE MEASUREMENT

20. Benchmark Summary: Baseline, Theoretical Optimisation, Empirical Test

Two tests quantify the effect of the scaffolding. The first is an **analytical estimate** on three representative tasks; the second is a **live measurement** run in Google Antigravity on real project files. Both compare the same two pipelines.

`Methodology`

- **Baseline pipeline:** a single unconstrained high-tier agent, no routing, no gate. It loads whole files to find context (including a ~350 KB source file, about 88,000 tokens by itself) and writes code or prose directly.
- **Optimised pipeline:** the architecture of this guide. Readiness gate first, surgical lookup (grep of one section, symbol index with offset/limit), a short Work Order from the planner, execution by a low-tier agent, and a deterministic verification command that ends the task with an explicit status.
- **Test 1, theoretical estimate:** three task types (a cross-file refactor, an ambiguous debug request without reproduction steps, and a documentation realignment). Token counts and times are analytical estimates from file sizes and typical agent behaviour, not measurements.
- **Test 2, empirical live test:** one documentation-consistency task (check one progress sheet against the rules of its area and validate repository integrity), executed twice on the real repository: once with full file reads, once through the optimised pipeline. Bytes, lines, tokens and wall-clock time were measured; the verification was the project's audit script.
- **Metrics:** bytes and lines read, input tokens, output tokens (high-tier and low-tier), elapsed time, share of a free-plan quota consumed, and whether the result was deterministically verified.

Test 1 - theoretical estimate, totals over the three tasks

Metric	Baseline	Optimised (estimate)	Diff
Input tokens	252,000	2,000	-99.2%
Output tokens, high-tier	11,100	245	-97.8%
Output tokens, low-tier	0	510	work delegated
Time	~270 s	~22 s	-91.8% (12x)
Completion rate	40% average	100%	gate blocks the ambiguous task
Errors / regressions	4	0	deterministic
Free-plan quota	exhausted by turn 2	~3.5% used	session continues

Test 2 - live measurement on real files

Metric	Baseline (measured)	Optimised (measured)	Diff
Bytes read	406,994	1,037	-99.7%
Lines loaded	10,039	16	-99.8%
Input tokens	107,103	404	-99.6%
Output tokens	578	96	-83.4%
Elapsed time	14.48 s	2.05 s	-85.8% (7x)
Final verification	none	DONE, audit passed	deterministic
Free-plan quota	session lock	0.4% used	negligible

In the baseline run the single turn already exceeds the free-plan context limit and the session is blocked. In the optimised run the gate routed the task to the planner, a 16-line lookup produced a 76-token Work Order, and the low-tier agent ran the audit script (no broken links) and closed with **DONE** in 20 tokens.

Three-way comparison for the documentation task

The table sets the baseline against the theoretical optimisation and the live result. The estimate was built for the documentation task of Test 1; the measured baseline comes from the Test 2 run, a similar task on a larger set of files, so its baseline figures are higher than the estimated baseline.

Parameter	Baseline (estimate)	Baseline (measured)	Optimised (estimate)	Optimised (measured)
Data read	whole documents	406,994 B	~1,500 B	1,037 B
Input tokens	~45,000	107,103	~450	404
Output tokens	~4,100	578	~90-100	96
Time	~95 s	14.48 s	~6 s	2.05 s
Verification	none (verbose, unverified)	none	audit expected OK	audit passed, DONE
Free-plan quota	not executable	session lock	~1%	0.4%

Optimised pipeline: estimate vs measurement

Parameter	Estimate	Measured	Deviation
Data read	~1,500 B	1,037 B (16 lines)	-30.8% (leaner)
Input tokens	~450	404	-10.2%
Output tokens, planner	~70-80	76	within range
Output tokens, mechanic	~20-30	20	within range
Output tokens, total	~90-100	96	within range
Time	~6.0 s	2.05 s	-65.8% (3x faster)
Outcome	audit OK expected	audit passed, DONE	confirmed
Free-plan quota	~1%	0.4%	-60%

Reading the results

- **The model is accurate where it matters:** total output tokens were estimated at 90-100 and measured at 96. Reads and input tokens came in lower than predicted, so the estimate is conservative.

- **Time is the loosest figure:** the estimate included a margin for cloud latency; local tool execution made the run about 3x faster than predicted.
- **The saving comes from reading less, not from a cheaper model alone:** input fell by about 99.6% because of the gate and the surgical lookup. The output saving comes from planning separated from execution.
- **Caveats:** the live test covers a single documentation task; the refactor and ambiguous-debug results of Test 1 remain estimates. Quota figures depend on the plan and can change.